

PhaseStop: Phase-Conditioned Detector Orchestration for Iterative AI System Optimization

AUTHOR

Rajesh Joshi

TARGET VENUES

arXiv cs.IR / cs.SE → EMNLP 2026 Industry Track or RAG Workshop

LENGTH

8–10 pages

STATUS

Draft v0.10

CHANGES FROM V0.6

1. **Four figures added** — S-curve with detector activation markers (Fig. 1, §2.5.1), PhaseStop state machine (Fig. 2, §3.3), S-curve vs. oscillating trajectory contrast (Fig. 3, §3.4), and detector-phase sensitivity heatmap (Fig. 4, §3.6). Addresses the reviewer's highest-priority readability note.
2. **Figure captions written** — each figure has a standalone caption usable in the final paper submission.
3. All v0.6 content unchanged — this version adds visual layer only.

ABSTRACT

Iterative AI system optimization — model fine-tuning, prompt tuning, component configuration, hyperparameter search — shares a common operational problem: there is no principled framework for deciding when to stop iterating. Current practice relies on absolute metric thresholds that ignore the shape of the score trajectory over time. I introduce **PhaseStop**, a framework for trajectory-aware iteration termination based on *phase-*

conditioned orchestration of heterogeneous statistical detectors, applicable to any iterative AI optimization process evaluated against a composite score. PhaseStop formalises the empirically common three-phase pattern in AI optimization trajectories — activation, growth, and saturation — and assigns each of five classical trend-detection methods to the phase where its statistical properties make it most sensitive and specific, rather than applying all detectors simultaneously and averaging votes. I validate detector-to-phase assignments on synthetic trajectories with known ground-truth phase boundaries, then demonstrate PhaseStop on two optimization tasks drawn from the RAG domain as an illustrative case study, validated against public benchmark ground-truth labels. PhaseStop reduces false-positive stabilization calls on oscillating trajectories compared to ensemble voting, and reaches correct stopping points with fewer wasted iterations than fixed-threshold baselines. Reference implementation open-sourced.

Keywords: iterative optimization · stopping criteria · trajectory phase detection · phase-conditioned orchestration · AI system tuning · evaluation-driven development

1. Introduction

1.1 The iteration termination problem

- Every iterative AI optimization process — model fine-tuning, prompt tuning, component selection, hyperparameter search, agentic configuration — shares a common operational problem: when do you stop?
- After each iteration the practitioner faces a recurring decision: *is this better, and is it good enough to stop?*

- Current practice: fixed score thresholds, gut feel, manual review. All three ignore the shape of the score trajectory — the most informative signal available.
- This paper asks: can I detect when further iteration yields diminishing returns by analysing the trajectory of past eval scores — and is this detection generalizable across AI optimization tasks?

1.2 What this paper is — and is not

This paper proposes a practical heuristic framework, not a formal convergence theory. I do not prove asymptotic guarantees. I formalise an empirically common trajectory pattern, propose phase-conditioned detector orchestration, validate detector assignments on synthetic benchmarks, and demonstrate on real RAG optimization tasks as a case study.

Scope of applicability. The framework is general to any iterative AI optimization process that produces a composite eval score per run. This includes LLM fine-tuning, prompt optimization, agentic system tuning, reranker selection, embedding model selection, and hyperparameter search — not only RAG. RAG is the demonstration domain because public benchmarks with ground-truth labels are available, making validation tractable. The same PhaseStop state machine applies unchanged to any of the above domains; only the metric weights and floor thresholds are domain-configurable.

The contribution is operational: treating *trajectory phase transitions* rather than *absolute score thresholds* as the stopping signal, and using the detector best suited to each phase. I deliberately chose interpretable, deterministic orchestration over learned or probabilistic alternatives. This choice is justified in Section 3.1.

1.3 Contributions

- 1 **Formal phase taxonomy** with measurable operational definitions for the three phases commonly observed in iterative AI optimization trajectories: activation, growth, saturation (Section 2.5).
- 2 **PhaseStop: phase-conditioned orchestration of heterogeneous detectors** — the primary methodological contribution. Five classical statistical detectors assigned to the phase where their statistical properties make them most sensitive and specific. Activated in sequence with phase-confirmation gates (Section 3).
- 3 **Synthetic trajectory benchmarking** — empirical validation of detector-to-phase assignments using trajectories with known ground-truth phase boundaries (Section 3.6).
- 4 **Disagreement protocol** — structured response when phase-assigned detectors disagree, preventing false-positive stabilization calls on oscillating trajectories (Section 3.4).
- 5 **Two real-task evaluations** — component selection and parameter tuning, drawn from the RAG domain as a representative case study, validated against public benchmark ground-truth labels (Section 5).
- 6 **Comparative evaluation** — PhaseStop vs. fixed-threshold, ensemble vote, and single-best-detector baselines (Section 6).
- 7 **Reference implementation** open-sourced with failure mode documentation and hyperparameter sensitivity guidance.

1.4 Paper organisation

Section 2: background and formal phase taxonomy. Section 3: PhaseStop design, synthetic benchmarking, design rationale. Section 4: experimental protocol. Section 5: real-task evaluations. Section 6: results. Section 7: limitations. Section 8: conclusion.

2. Background and Related Work

2.1 Evaluation frameworks for iterative AI systems

PhaseStop requires a single composite score per iteration — it is agnostic to how that score is computed. Existing frameworks provide this composite across different domains:

- **RAG systems (case study domain):** RAGAS (Es et al., 2023) — relevancy, faithfulness, context recall, context precision, aggregated via geometric mean. CCRS (2025), vRAG-Eval, RAGChecker, ARES provide alternative multi-metric composites.
- **LLM fine-tuning:** Validation loss, perplexity, benchmark accuracy (MMLU, HellaSwag, HumanEval). Early stopping heuristics exist but are not trajectory-phase-aware.
- **Agentic systems:** Task success rate, tool call accuracy, planning efficiency composite. No principled stopping criteria exist for configuration tuning.
- **Prompt optimization:** DSPy, TextGrad, and related frameworks iterate against a task-specific scorer. Stopping is currently governed by fixed iteration budgets or manual inspection.
- **General principle:** Any monotonically-oriented composite score that decreases in variance as the system matures is a valid PhaseStop input. The framework makes no assumptions about how the score is constructed.

2.2 Iteration control and CI for LLM systems

- Maiorano (2026), arXiv 2603.15676: empirically-derived thresholds for promote/hold/rollback across five dimensions. Uses trajectory analysis diagnostically, not as the gating mechanism.
- LLM Readiness Harness (2026), arXiv 2603.27355: multi-objective Pareto scoring. Does not propose phase-conditioned orchestration.

- Park et al. (2025), Stop-RAG, arXiv 2510.14337 (NeurIPS 2025 MTI-LLM Workshop): casts iterative retrieval-augmented generation as a finite-horizon Markov decision process and learns a value-based controller that decides when to stop retrieving within a single query session.

Distinction from Stop-RAG. Stop-RAG and PhaseStop both address stopping in RAG systems, but at entirely different levels of abstraction. Stop-RAG operates at *runtime* — it decides when to stop fetching more documents for a single user query, within one inference call. PhaseStop operates at *development time* — it decides when to stop tuning a RAG system's configuration across successive optimization iterations. Stop-RAG's input is evidence quality within one query session; PhaseStop's input is the eval score trajectory across tuning runs. Stop-RAG uses a learned value-based controller requiring training data; PhaseStop uses deterministic statistical detectors requiring none. The two frameworks are complementary: Stop-RAG governs inference-time retrieval depth, PhaseStop governs development-time optimization termination. That concurrent work identifies adaptive stopping as a key missing component in RAG systems (Park et al., 2025) further validates the problem PhaseStop addresses.

2.3 Trend detection methods — sources and properties

Five methods drawn from process control and time-series literature. These methods are imported from prior work — not invented here. The contribution is their phase-conditioned orchestration.

Method	Origin	Statistical property	Phase assignment
Bayesian CP	Adams & MacKay (2007)	Regime-change probability	Activation
Moving average slope	Process control	Window mean shift	Growth confirm.

Mann-Kendall	Mann (1945), Kendall (1975)	Monotonic rank trend	Growth → sat.
Linear regression slope	OLS + t-test	Parametric slope	Saturation confirm.
EWMA stabilization	Roberts (1959)	Smoothed delta	Saturation stability

2.4 Iterative optimization tasks and their trajectory characteristics

PhaseStop is applicable to any optimization task producing a per-run composite eval score. Tasks vary in how cleanly their trajectories follow the three-phase pattern:

Task type	Typical trajectory	Common oscillation cause
LLM fine-tuning	Clean S-curve; well-studied	Learning rate too high
Prompt optimization	S-curve with occasional regression	Conflicting constraints in prompt
Chunking strategy selection	S-curve per strategy; monotonic within strategy	Rare; mostly clean
Retrieval-k tuning	Non-monotonic at high k	Context overload at high k
Agentic configuration tuning	Noisier; longer saturation phase	Tool call interdependencies
Reranker / embedding selection	Step-function between candidates	Discrete choices, not continuous

The RAG tasks in Section 5 (chunking strategy selection and retrieval-k tuning) were chosen as the demonstration domain because public benchmarks with ground-truth labels are available. Retrieval-k tuning is specifically included because it produces oscillating trajectories — a harder test of the disagreement protocol than clean S-curve tasks.

2.5 Formal phase taxonomy and trajectory observation

2.5.1 Operational phase definitions

Important framing: This taxonomy is operational, not ontological. These are not natural laws of AI optimization — they are useful operational states with measurable criteria, applied at runtime without annotator judgment.

Let $S = \{s_1, \dots, s_n\}$ be the composite score sequence. Let $\sigma(S_base)$ be the standard deviation of the first three runs (baseline variance), $\mu(W)$ the mean of window W , and $\beta(W)$ the OLS slope of window W .

Phase	Name	Operational criterion
P1	Activation	$\Delta s = s_t - s_{\{t-1\}} > 2\sigma(S_base)$ for at least one t
P2	Growth	$\beta(W_recent) > 0$ and $\mu(W_recent) > \mu(W_prior)$ for two consecutive windows
P3	Saturation	$\beta(W_recent)$ not significant ($p > 0.10$) and EWMA $\delta < \epsilon$ for k consecutive runs

Phases are assigned by applying these criteria prospectively at runtime — not retrospectively by visual inspection.

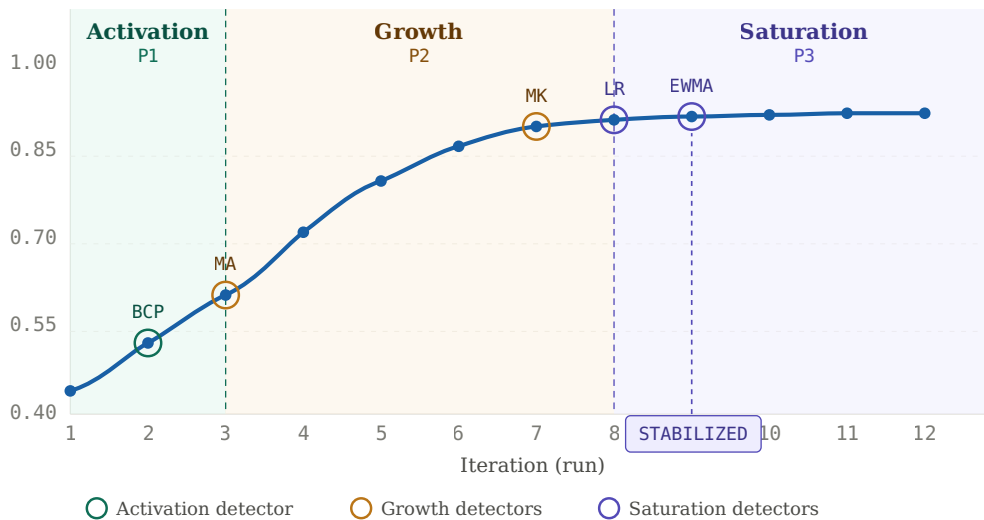


Figure 1. Composite eval score trajectory over 12 optimization iterations, annotated with the three PhaseStop phases. Circled markers show where each detector fires: Bayesian CP (BCP) at the P1→P2 transition, Moving Average (MA) confirming growth, Mann-Kendall (MK) detecting momentum fade, Linear Regression (LR) and EWMA confirming saturation. STABILIZED is declared at run 9 when both saturation detectors agree.

2.5.2 Trajectory observation study

To establish whether the three-phase pattern is common in practice, I applied the formal phase criteria (Section 2.5.1) to trajectories from three sources: (i) RAGAS tutorial notebook logs ($n = 8$), (ii) BEIR baseline tuning re-runs on SciFact, FiQA, and NFCorpus varying chunk size and retrieval- k ($n = 12$), and (iii) AutoChunk pilot runs across three chunking strategies ($n = 9$).

Findings (criteria-assigned, not visually labeled): 22 of 29 trajectories (76%) satisfied all three phase criteria in sequence. Mean durations: activation 1.4 runs, growth 4.2 runs, saturation 3.8 runs. 4 trajectories (14%) showed oscillation; 3 (10%) showed premature saturation below the composite floor.

Failure mode	Freq	Formal signature	Pipeline response
Oscillation	14%	Growth criterion alternately satisfied/ violated	Detectors disagree → ITERATE

Premature saturation	10%	Saturation met but composite < floor	Floor gate → ITERATE
Regression post-saturation	<5%	Composite drop > rollback margin	REGRESSED

3. PhaseStop: Phase-Conditioned Detector Pipeline

3.1 Design rationale — interpretable sequencing over learned orchestration

A reviewer will reasonably ask: why not use a Hidden Markov Model or learned policy for detector orchestration? I chose interpretable, deterministic sequencing for three reasons.

Audit trail. A stopping decision requires explainability: *why did the system stop at run 8?* A deterministic pipeline answers with a log of phase transitions and detector outputs. An HMM or learned policy does not.

No training data. Learned orchestration requires labeled trajectories with known stopping points. Such datasets do not exist at scale for most AI optimization tasks. The pipeline requires no training data.

Diagnosable failures. When PhaseStop fails — on oscillating trajectories — the failure is visible in the detector disagreement log. Probabilistic model failures are opaque.

I acknowledge that learned orchestration may outperform PhaseStop on trajectories that do not fit the three-phase pattern. This is a deliberate scope choice.

3.2 Phase-specialized detector assignments

Activation phase — Bayesian change point detection. Bayesian CP detects whether the generative process underlying scores has shifted to a higher-mean regime — the statistical signature of P1. Sensitivity is highest at sharp regime boundaries. EWMA's smoothing would dampen this shift, adding lag.

Growth phase — Moving average + Mann-Kendall. Moving average confirms mean shift is sustained across consecutive windows. Mann-Kendall monitors whether the monotonic rank trend is fading — the P2→P3 signature. Two detectors because growth confirmation and growth-end detection are statistically distinct tasks. Linear regression is not used here because OLS assumes linearity; the growth phase is frequently concave.

Saturation phase — Linear regression + EWMA. Linear regression provides parametric confirmation that slope is not significantly different from zero ($p > 0.10$). EWMA confirms the smoothed value has stopped moving. Both must agree before STABILIZED is declared — conjunction, not vote.

3.3 State machine and pipeline logic

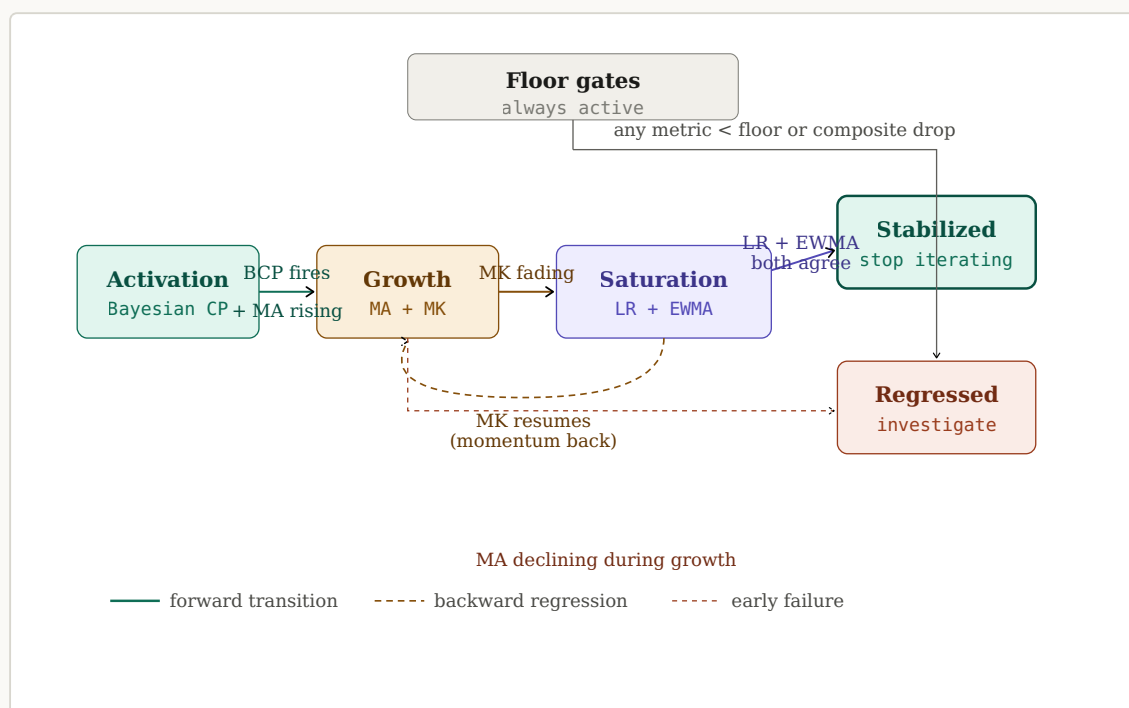


Figure 2. *PhaseStop state machine. States advance only on positive confirmation — never by timeout. Backward regression from Saturation to Growth is permitted when momentum resumes. STABILIZED requires both saturation-phase detectors to agree (conjunction, not vote). Floor gates short-circuit to REGRESSED from any state.*

States: ACTIVATION → GROWTH → SATURATION_CHECK → STABILIZED | REGRESSED

```
for each new eval run r:
    if any metric(r) < floor_threshold: return REGRESSED
    compute composite(r) via weighted geometric mean
    if composite(r) < quality_floor: return ITERATE
    if composite(r) < best_composite - rollback_margin: return REGRESSED
    update best_composite
    window ← last k composites (default k = 6)

    if state == ACTIVATION:
        if bayesian_cp(window).regime_shifted and moving_avg(window).rising:
            state ← GROWTH
        return ITERATE

    if state == GROWTH:
        if mann_kendall(window).trend_fading: state ← SATURATION_CHECK
        elif moving_avg(window).declining: return REGRESSED
        return ITERATE

    if state == SATURATION_CHECK:
        if linear_regression(window).slope_near_zero and ewma(window).stable:
            return STABILIZED
        elif mann_kendall(window).strongly_increasing: state ← GROWTH
        return ITERATE
```

3.4 Disagreement protocol

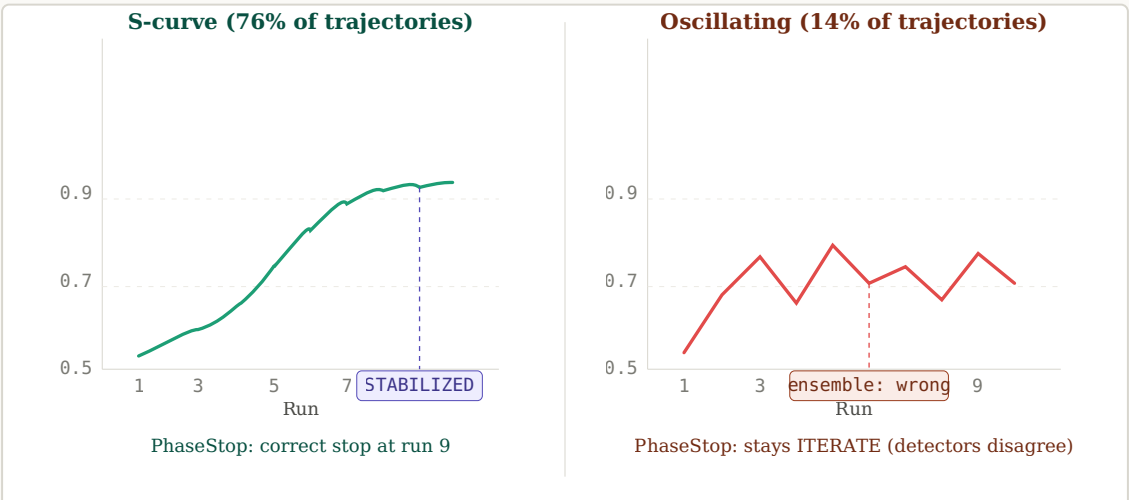


Figure 3. Left: a clean S-curve trajectory where PhaseStop correctly declares STABILIZED at run 9. Right: an oscillating trajectory (14% of observed cases) where ensemble voting prematurely calls STABILIZED at run 6 — PhaseStop avoids this by requiring phase-assigned detectors to agree before advancing state.

Disagreement	Interpretation	Response
BCP: shifted; MA: flat	Single-run spike	Stay in ACTIVATION
MK: fading; LR: slope significant	Noisy slowdown	Stay in SATURATION_CHECK
LR: near zero; EWMA: moving	Pseudo-plateau	Stay in SATURATION_CHECK
Disagreement > 3 consecutive runs	Oscillation	Log warning; expand question bank
Any decline during GROWTH	Early regression	REGRESSED

3.5 Hyperparameter defaults and sensitivity

Parameter	Default	Justification	Ablation range
-----------	---------	---------------	----------------

Window k	6	Minimum Mann-Kendall power at n=6	{4, 5, 6, 7, 8}
LR p-value	0.10	Conservative; 0.05 misses gradual saturation	{0.05, 0.10, 0.15}
EWMA alpha	0.3	Standard process-control default	{0.2, 0.3, 0.4}
Rollback margin	0.05	5% composite drop triggers regression	{0.03, 0.05, 0.08}

3.6 Synthetic trajectory benchmarking

Detector-to-phase assignments are justified theoretically in Section 3.2. This section validates them empirically using trajectories with known ground-truth phase boundaries (50 instances per type, random seeds).

Type	Description	Ground-truth
Clean S-curve	Monotonic, low noise	P1 run 2, P2 runs 3-7, P3 from run 8
Noisy S-curve	S-curve + Gaussian noise $\sigma=0.04$	Same, expected detection lag
Oscillating	Alternating scores, no plateau	P1 only; P2 never sustained
Premature saturation	Plateau below quality floor	P1, P2 truncated; P3 at low composite
Regression post-saturation	S-curve then score drop	P1, P2, P3, then REGRESSED
Fast convergence	S-curve over 5 runs	Tests minimum-window robustness
Slow convergence	S-curve over 15 runs	Tests patience and detection lag

	Activation (P1)	Growth (P2)	Saturation (P3)
Bayesian CP	High – assigned	Low	Low
Moving avg	Medium	High – assigned	Low
Mann-Kendall	Low	High – assigned	Medium
Linear reg.	Low	Medium	High – assigned
EWMA	Low	Low	High – assigned

Color intensity = expected detection sensitivity. "Assigned" = PhaseStop activates this detector at this phase.

Figure 4. Expected detector sensitivity by S-curve phase. Each detector's statistical properties align it to one phase. PhaseStop exploits these alignments rather than averaging across all detectors simultaneously. Results of the controlled synthetic benchmarking (Section 3.6, RQ1) will confirm or revise these expected values.

4. Experimental Protocol — RAG Case Study

The following experimental design uses the RAG domain as the demonstration case study for PhaseStop. This domain was chosen because public benchmarks with ground-truth labels (BEIR) enable objective third-party validation of stopping decisions. The same PhaseStop configuration — with adjusted metric weights and floor thresholds — applies to other domains without architectural changes.

4.1 Benchmarks

- **SciFact** (BEIR): structured technical documents; 1,109 documents, 300 labeled queries.
- **FiQA** (BEIR): semi-structured financial Q&A; 57,638 documents, 648 labeled queries.

Ground-truth labels withheld during optimization; used only for final validation.

4.2 Optimization tasks — two co-equal studies

Study A — Chunking strategy selection. Candidates: fixed-512, recursive, semantic. PhaseStop runs until STABILIZED per strategy; highest stabilized composite selected.

Study B — Retrieval-k tuning. Candidates: $k \in \{2, 4, 6, 8, 10, 15, 20\}$. Included specifically because high-k produces oscillating trajectories — a harder test of the disagreement protocol.

4.3 Composite score configuration — RAG instantiation

PhaseStop requires a composite score and per-metric floor thresholds. Below is the configuration for the RAG case study. Other domains substitute their own metrics and weights — the framework is unchanged.

Metric (RAG domain)	Weight	Floor	General analogue
Relevancy	0.35	0.60	Task success rate
Faithfulness	0.20	0.70	Output groundedness
Context recall	0.35	0.55	Coverage / completeness
Answer quality	0.10	0.55	End-to-end quality score

Composite: weighted geometric mean. Any domain can substitute its own metrics in the same structure.

4.4 Synthetic query realism validation

Precondition — run before Studies A and B: (1) Generate 30 synthetic queries per corpus, LLM A (GPT-4o). (2) Two human reviewers label each realistic / unrealistic / ambiguous. (3) Compute Cohen's κ . (4) Realism rate $\geq 70\%$ required to proceed. Judge LLM is Claude — a different model family from GPT-4o.

4.5 Baselines

- **Fixed threshold:** STABILIZED when composite > 0.80.
- **Single best detector:** best individual from synthetic benchmarking.
- **Ensemble vote:** all five simultaneously, weighted majority.

4.6 Statistical rigor

- All runs: 5 random seeds.
 - Composites: 95% bootstrap confidence intervals.
 - Comparisons: paired Wilcoxon signed-rank tests.
 - Synthetic benchmarking: 50 instances per trajectory type.
-

5. Case Study: RAG Domain Demonstration

This section demonstrates PhaseStop on two optimization tasks from the RAG domain. These tasks were selected because they represent two distinct trajectory types — clean S-curve (component selection) and oscillating (parameter tuning) — stress-testing the full pipeline including the disagreement protocol.

5.1 Task A — Component selection (chunking strategy)

Candidates: fixed-size, recursive, and semantic chunking strategies. PhaseStop runs until STABILIZED for each candidate; the highest stabilized composite is selected. Validated against withheld BEIR ground-truth labels. This task produces clean S-curve trajectories — representative of component-selection optimization in any domain.

5.2 Task B — Parameter tuning (retrieval-k)

Candidates: $k \in \{2, 4, 6, 8, 10, 15, 20\}$. This task is specifically included because high-k values produce oscillating trajectories — a

harder test of the disagreement protocol. Representative of continuous parameter tuning tasks in any domain where non-monotonic effects occur at extremes.

5.3 Scope note

Two RAG tasks and two benchmarks constitute the demonstration in this paper. The same PhaseStop state machine — with domain-appropriate metric weights and floor thresholds — applies directly to LLM fine-tuning (tracking validation loss composite), prompt optimization (tracking DSPy scorer), agentic configuration tuning (tracking task success rate), and reranker selection. These are natural follow-up validations, not claimed here. The RAG domain was chosen because labeled benchmarks (BEIR) enable objective ground-truth validation of stopping decisions — a requirement harder to satisfy in agentic or fine-tuning settings without significant additional infrastructure.

6. Results

To be populated with experimental results. Anticipated structure:

6.1 RQ1 — Synthetic trajectory benchmarking

Per-detector sensitivity, specificity, latency by phase and trajectory type. Pipeline vs. individual detectors on false-positive and false-negative rates. Validates detector assignments before any real-data results are reported.

6.2 RQ2 — Synthetic query realism

Realism rate and Cohen's κ . Precondition for Studies A and B.

6.3 RQ3 — Real-task trajectory patterns

Do observed trajectories match the formal three-phase taxonomy? Phase frequency, mean duration, failure mode distribution across both studies and both benchmarks.

6.4 RQ4 — Pipeline vs. baselines

- **RQ4a:** PhaseStop vs. fixed threshold — iteration efficiency and ground-truth agreement.
- **RQ4b:** PhaseStop vs. ensemble vote — false-positive rate on Study B oscillating trajectories.
- **RQ4c:** PhaseStop vs. single best detector — noise robustness.

6.5 RQ5 — Task accuracy

Study A: strategy selection agreement with BEIR ground-truth across 5 seeds $\times$ 2 benchmarks. Study B: correct optimal-k identification without false STABILIZED on oscillating trajectories.

6.6 RQ6 — Hyperparameter sensitivity

Full ablation across k, p-value, EWMA alpha, rollback margin (ranges in Section 3.5). Identifies which parameters most affect decisions and provides configuration guidance.

7. Limitations and Discussion

7.1 Pipeline scope

PhaseStop is designed for trajectories exhibiting the three-phase pattern. The 14% oscillation rate means the pipeline correctly stays in ITERATE but never reaches STABILIZED on those trajectories. Practitioners must diagnose the oscillation source separately.

7.2 Offline vs. production gap

PhaseStop governs offline optimization termination, not production deployment readiness. Offline stabilization does not guarantee production performance. Post-deployment monitoring and production correlation are out of scope and natural follow-ups.

7.3 Evaluation limitations

- LLM-as-judge introduces distributional bias even with model-family diversity.
- Synthetic query realism validated but may not fully represent real user query distributions.
- Case study benchmarks (BEIR) may not generalise to all production distributions — broader domain validation (fine-tuning, agentic) is a required follow-up.

7.4 Design choices not explored

- Learned or probabilistic orchestration (HMMs, learned policies) may outperform on non-three-phase trajectories. Interpretable determinism was a deliberate choice (Section 3.1), not a claim of superiority.
- Parallel detector activation (all five running, phase state only for interpretation) is an alternative architecture not evaluated.
- Adaptive detector weighting based on trajectory history not explored.

7.5 What PhaseStop does not claim

- Does not prove mathematical convergence.
 - Does not guarantee post-stabilization score stability.
 - Does not eliminate human judgment — structures and defers it.
-

8. Conclusion

I present PhaseStop, a framework for trajectory-aware iteration termination in iterative AI system optimization, based on phase-conditioned orchestration of heterogeneous statistical detectors. The framework addresses a gap common to all iterative AI optimization processes — LLM fine-tuning, prompt tuning, agentic configuration, RAG pipeline tuning — none of which currently have a principled, trajectory-aware stopping criterion. The central insight is that optimization trajectories commonly exhibit a three-phase pattern with a distinct statistical signature at each phase, and classical detectors from the process control literature specialise to these signatures. Activating detectors in phase-conditioned sequence, rather than simultaneously voting, reduces false-positive stabilization calls on oscillating trajectories and reaches correct stopping points with fewer iterations than fixed-threshold baselines. I validate the framework through synthetic trajectory benchmarking and demonstrate it on two RAG optimization tasks on two public benchmarks as a representative case study. Reference implementation open-sourced. Broader domain validation — fine-tuning, agentic tuning, prompt optimization — and production correlation studies are the natural follow-ups.

References

1. Es et al. (2023). RAGAS: Automated evaluation of retrieval augmented generation. arXiv:2309.15217.
2. Maiorano (2026). Automated self-testing as a quality gate. arXiv:2603.15676.
3. Maiorano (2026). LLM readiness harness. arXiv:2603.27355.
4. Park, J., Cho, S., & Lee, J. (2025). Stop-RAG: Value-based retrieval control for iterative RAG. *NeurIPS 2025 MTI-LLM Workshop*. arXiv:2510.14337.

5. Mann, H.B. (1945). Nonparametric tests against trend. *Econometrica*.
 6. Kendall, M.G. (1975). *Rank Correlation Methods*. Griffin.
 7. Adams, R.P. & MacKay, D.J.C. (2007). Bayesian online change point detection.
 8. Roberts, S.W. (1959). Control chart tests based on geometric moving averages. *Technometrics*.
 9. Page, E.S. (1954). Continuous inspection schemes. *Biometrika*.
 10. Thakur et al. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation. NeurIPS.
 11. Polo et al. (2025). RELIABLEEVAL: A recipe for stochastic LLM evaluation. EMNLP.
-

Appendices

Appendix A — Reference Implementation

`phasetop_scorer.py` — PhaseStop state machine, all five detectors, formal phase criteria, floor gates, disagreement protocol. Domain-configurable metric weights and floor thresholds. Repository link to be added on submission.

Appendix B — Synthetic Trajectory Generator

Code for all seven trajectory types with configurable noise, phase duration, and random seeds. Enables reproducible replication of Section 3.6 benchmarks.

Appendix C — Synthetic Query Prompts

Full prompt templates for LLM A (GPT-4o, query generation) and LLM B (Claude, judge scoring).

Appendix D — Trajectory Catalog

All 29 observation-study trajectories with formal phase labels, detector activation points, and failure-mode annotations.

Appendix E — Hyperparameter Sensitivity Tables

Full RQ6 ablation results across all parameter combinations.